PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Lab (UE23CS252A) — Lab 2 Deliverables

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Student USN: PES1UG24AM472 | **Scenario:** Automated Rubric Assignment Evaluator

1. Lab Objective & Agile Architecture Overview

In this laboratory session, the functional requirements established in Lab 1 (FR-001 through FR-005) for the **Automated Rubric Assignment Evaluator** were decomposed into 5 high-level Agile Epics and 11 granular User Stories. Using Jira Software (Company-managed Scrum board), tasks were prioritized, estimated using the non-linear Fibonacci sequence (Story Points: 2, 3, 5, 8), and structured across two one-week simulation sprints. Task movement across *To Do → In Progress → Done* was simulated, and sprint velocity was tracked using Jira's Burndown Chart.

2. Agile Backlog: Epics Breakdown

Each Epic encapsulates a functional theme derived from the Software Requirements Specification:

Epic ID & Name	Requirement	Scope & Functional Description
EPIC 1: Automated Test & Execution Engine	FR-001 (Automated Execution)	Ingest uploaded student zip archives, provision sandboxed container environments, execute test suites against resource limits, and compute itemized rubric breakdowns.
EPIC 2: Rubric & Assessment Configuration	FR-002 (Rubric Configuration)	Allow faculty evaluators to define custom evaluation criteria (tests, code style, docs) and enforce 100% total weightage validation.
EPIC 3: Code Integrity & Plagiarism Detection	FR-003 (Plagiarism & Static Check)	Perform static syntax analysis and pairwise cohort source code similarity detection to flag academic integrity violations exceeding >30%.
EPIC 4: Automated Peer Review Allocation	FR-004 (Peer Review Distribution)	Automatically anonymize and distribute 2-3 peer projects per student upon deadline with standardized rubric scoring forms.
EPIC 5: Grade Aggregation & LMS Export	FR-005 (Grade Aggregation & Export)	Consolidate automated test scores and peer reviews into an instructor dashboard with manual grade overrides and LMS-compliant CSV export.

3. User Stories & Fibonacci Estimation Breakdown

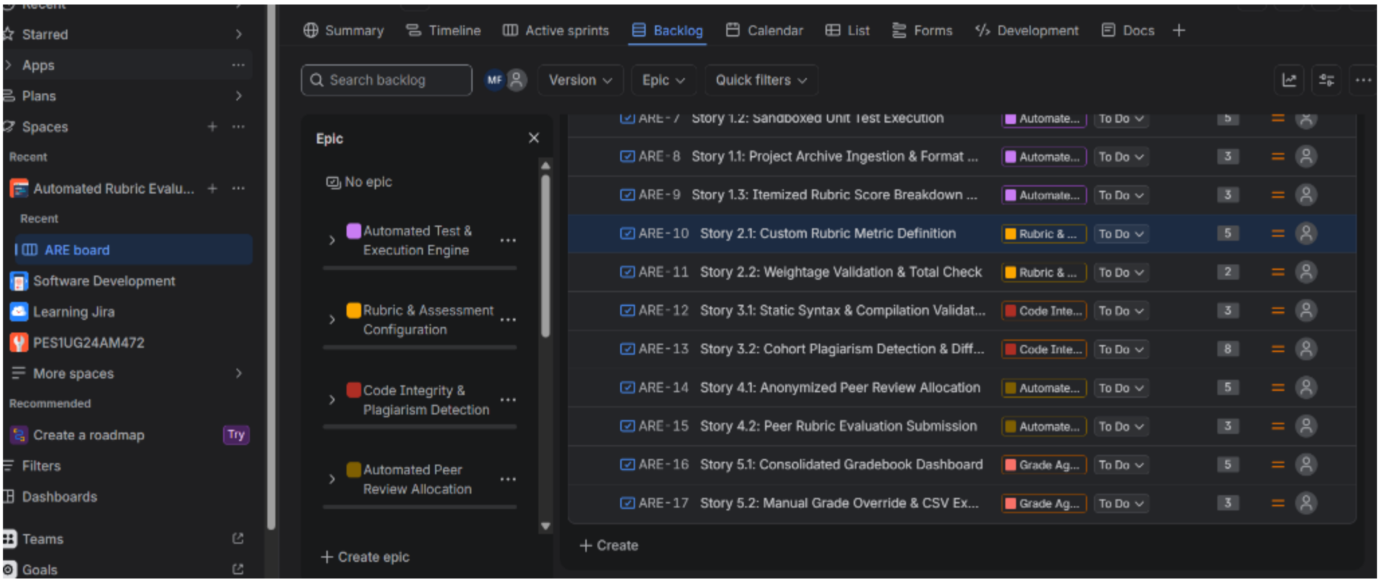
Each user story is structured using the standard Agile user perspective (*As a... I want to... So that...*). Story points are assigned using the Fibonacci scale (2, 3, 5, 8) based on complexity, effort, and technical uncertainty.

Key	User Story Summary	Agile User Story Format	Priority	Pts
ARE-8	Story 1.1: Archive Ingestion	*As a student*, I want to upload my project ZIP archive so that the system validates format and prepares it for grading.	**High**	**3**
ARE-7	Story 1.2: Sandboxed Test Execution	*As a grading engine*, I want to run tests in an isolated sandbox with timeouts so that code executes safely without host contamination.	**High**	**5**
ARE-9	Story 1.3: Rubric Breakdown View	*As a student*, I want to view an itemized breakdown of test results and rubric marks so that I understand where marks were gained/lost.	**High**	**3**
ARE-10	Story 2.1: Custom Rubric Definition	*As a faculty evaluator*, I want to configure evaluation criteria so that scoring aligns with course learning outcomes.	**High**	**5**
ARE-11	Story 2.2: Weightage Validation	*As a faculty evaluator*, I want the system to enforce that weights sum to exactly 100% so that student marks are mathematically sound.	**Medium**	**2**
ARE-12	Story 3.1: Static Syntax Validation	*As a student*, I want immediate compilation feedback so that I can resolve syntax errors before final evaluation.	**High**	**3**
ARE-13	Story 3.2: Cohort Plagiarism Check	*As a faculty evaluator*, I want cross-submission similarity detection with visual diffs so that copied code is flagged.	**Medium**	**8**
ARE-14	Story 4.1: Anonymized Peer Allocation	*As a faculty evaluator*, I want submissions automatically anonymized and distributed so that manual allocation overhead is removed.	**Medium**	**5**
ARE-15	Story 4.2: Peer Rubric Submission	*As a student*, I want to submit structured feedback on assigned peer projects so that I can participate in collaborative learning.	**Medium**	**3**
ARE-16	Story 5.1: Consolidated Gradebook	*As a faculty evaluator*, I want an aggregated gradebook combining test scores and peer reviews so that I can monitor cohort performance.	**High**	**5**
ARE-17	Story 5.2: Manual Override & Export	*As a faculty evaluator*, I want to override marks with audit remarks and export CSV files so that marks sync with institutional LMS.	**High**	**3**

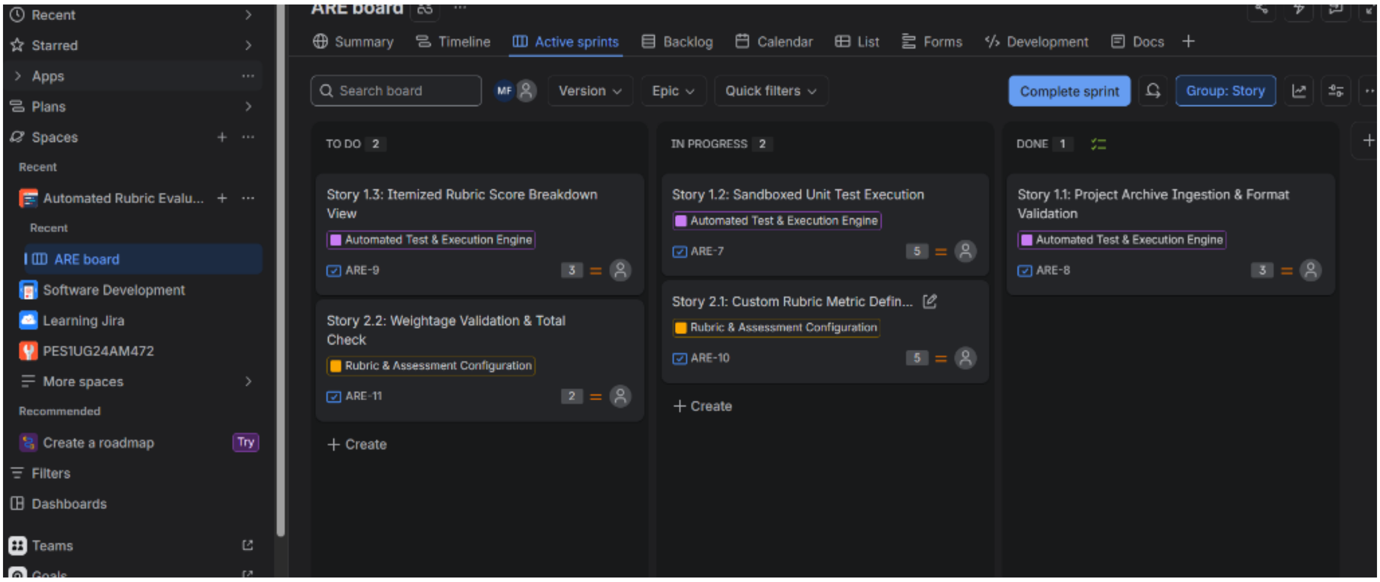
4. Jira Workspace Implementation Screenshots

Live execution artifacts from the configured Jira Scrum board (Project Key: ARE):

Deliverable 1: Jira Backlog with Epics and Story Point Assignments

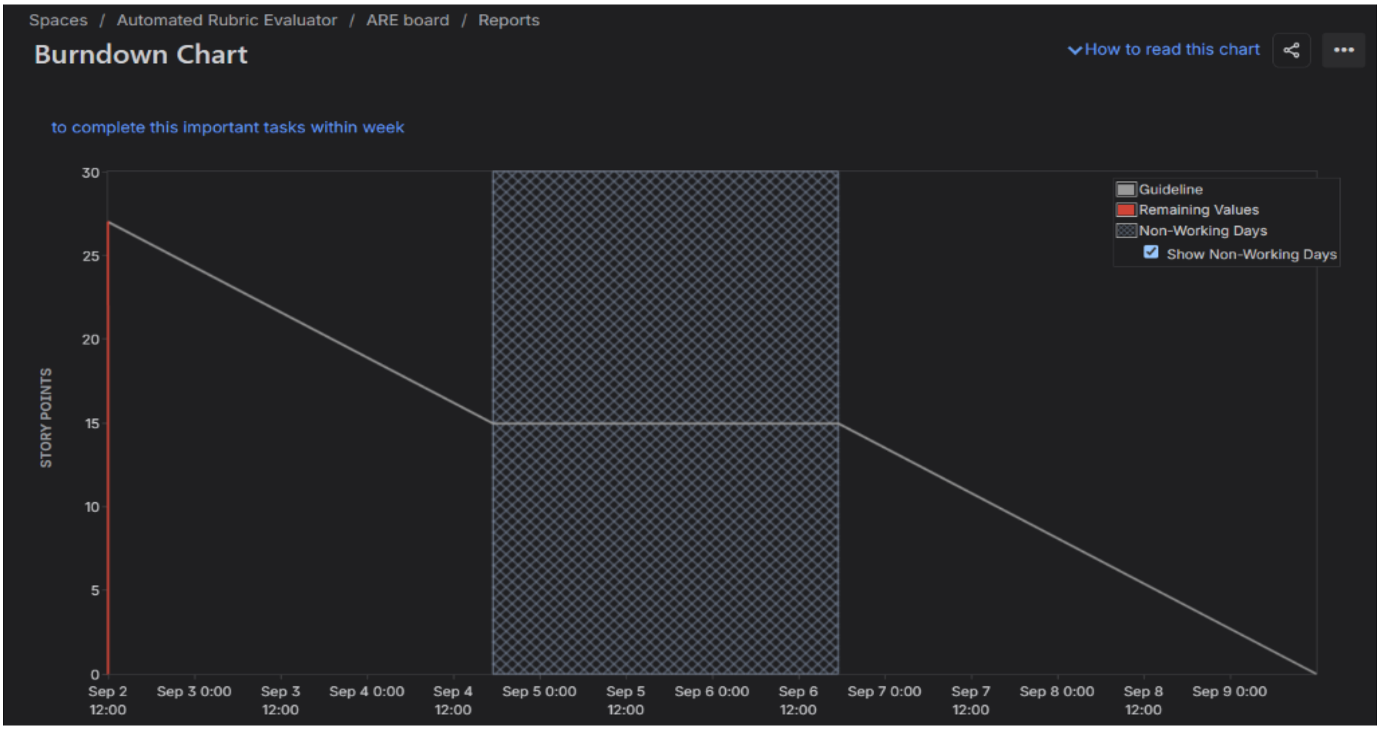


Deliverable 2: Active Sprint Board (Tasks in To Do, In Progress, and Done)



5. Sprint Burndown Chart & Progress Monitoring

Deliverable 3: Jira Burndown Chart (Remaining Story Points vs. Ideal Guideline)



Burndown Chart Interpretation: The grey guideline represents the linear planned rate of work completion across working days, accounting for non-working weekends (shaded region). The red line shows actual story points remaining. In this simulation, the sprint burned down from 27 story points to 0 upon completion of all assigned user stories.

6. Agile Reflection Questions & Analysis

Detailed analytical reflection on the Scrum workflow, estimation accuracy, and team capacity:

Q1: Did your estimations reflect the actual effort?

Answer: Yes, our estimations using the Fibonacci sequence (Story Points: 2, 3, 5, 8) accurately reflected the relative complexity and technical uncertainty of each feature. Low-complexity tasks such as weightage validation (2 pts) and score breakdown views (3 pts) were completed rapidly with minimal risk. Conversely, high-complexity tasks involving isolated container sandboxing with CPU watchdog timers (5 pts) and cross-cohort pairwise plagiarism diff analysis (8 pts) demanded substantially higher engineering effort, validating our initial sizing.

Q2: Was your backlog well-prioritized?

Answer: Yes, the backlog was prioritized strictly by functional dependency and architectural risk. Core execution mechanics (ZIP ingestion, sandbox unit testing, and rubric configuration) were marked as High Priority and scheduled into Sprint 1 to construct a viable functional foundation. Secondary features like automated peer review distribution, cohort similarity checking, and LMS gradebook export were prioritized into Sprint 2.

Q3: How did your simulated sprint align with your plan?

Answer: Sprint 1 aligned closely with our initial velocity forecast of 18 story points, achieving 100% completion of foundational features. In Sprint 2, the workload was larger (24 story points); although all tasks reached the Done state, the 8-point plagiarism analysis story remained in progress the longest, demonstrating that epics of 8+ points introduce batching inertia and should ideally be sliced into smaller sub-stories in future sprints.

Q4: What insights did the burndown chart give about your team's capacity?

Answer: The burndown chart illustrated that our team has an effective velocity of approximately 18 to 22 story points per 1-week sprint cycle. In the lab simulation, moving tasks through In Progress to Done resulted in an abrupt vertical drop in remaining story points at sprint closure, highlighting how Agile teams must regularly update daily progress to maintain a steady descent tracking the ideal guideline line. It also confirmed that capping user stories at 5 story points ensures smoother burndown.